

Functional SlowHeat

Plasticidade por neurônio, baselines publicados e resultados preliminares em Split-MNIST e Permuted-MNIST

$$u_i = |z_i \cdot \partial L / \partial z_i|$$

medir utilidade



consolidar evidencia



reservar capacidade



mascarar o update final

31 configuracoes · 2 benchmarks · 10 seeds

Estabilidade e plasticidade precisam coexistir

Quando tudo permanece plastico

Tarefas novas sobrescrevem caminhos uteis. O modelo aprende o presente e perde desempenho nas tarefas anteriores.

Quando tudo e protegido

A rede preserva o passado, mas bloqueia a aquisicao de tarefas novas. Retencao sem plasticidade tambem falha.

Pergunta central

E possivel proteger seletivamente neuronios importantes e, ao mesmo tempo, garantir uma fracao minima de capacidade livre?

Os benchmarks isolam dois tipos diferentes de interferencia

Split-MNIST · class-incremental

- 5 tarefas: (0,1), (2,3), (4,5), (6,7), (8,9)
- Uma unica cabeca de 10 classes; sem task ID no teste principal
- MLP 784 → 256 → 128 → 10; 10 epocas por tarefa
- Principal risco: competicao e calibracao entre tarefas

Permuted-MNIST · domain-incremental

- 5 dominios com permutacoes fixas dos 784 pixels
- Todas as tarefas usam as mesmas 10 classes
- MLP 784 → 512 → 256 → 10; 10 epocas por dominio
- Principal risco: preservar representacoes entre dominios

Pareado por seed · mesma inicializacao e minibatches · IC95% normal · n = 10

SlowHeat converte utilidade em protecao seletiva

1

Medir

Acumula $|z \cdot \partial L / \partial z|$ por unidade e normaliza dentro da camada.

2

Consolidar

Na fronteira da tarefa, combina a evidencia persistente por max, media ou soma.

3

Reservar

Um ranking limita quantas unidades ficam protegidas; o budget preserva plasticidade.

4

Aplicar

A mascara fatorada protege a linha de entrada e as colunas de saida do neuronio.

$$m_i = 1 / (1 + \beta \cdot \text{slow_heat}_i) \quad \cdot \quad M_i[i,j] = \min(m_destino[i], m_origem[j])$$

A mascara atua no delta final - inclusive no weight decay



Por que isso importa: escalar apenas o gradiente bruto pode ser parcialmente cancelado pela normalização do AdamW; o decay desacoplado também moveria parâmetros protegidos.

As 31 configuracoes pertencem a quatro familias

Controles

vanilla · hard_freeze · slowheat_none · reducao global de LR

Separam ganho real de efeitos de wiring, congelamento e taxa de aprendizado.

Memoria e logits

Replay · DER++ · ER-ACE · A-GEM

Usam memoria episodica para reintroduzir informacao antiga e controlar interferencia.

Regularizacao

SlowHeat · EWC · SI

Protegem parametros ou unidades usando importancia acumulada, sem guardar imagens antigas.

Distillation e hibridos

LwF · distillation · SlowHeat + Replay/DER++

Preservam respostas antigas e combinam estabilidade funcional com memoria.

Replay e DER++ fornecem o sinal antigo que o fluxo atual nao contem

Replay

- Armazena 20 exemplos por classe em memoria episodica.
- Mistura exemplos atuais e antigos no mesmo step.
- Evita depender apenas de uma penalidade sobre parametros.

[Chaudhry et al. \(2019\) · Tiny Episodic Memories](#)

DER++

$$L = CE_atual + \alpha \cdot MSE(logits) + \beta \cdot CE_replay$$

- Guarda imagens, rotulos e logits produzidos no passado.
- O MSE preserva as respostas escuras; a CE ancora os rotulos.
- Neste projeto, $\alpha = 0,5$ e $\beta = 0,5$.

[Buzzega et al. \(NeurIPS 2020\) · DER++](#)

ER-ACE restringe classes; A-GEM projeta gradientes

ER-ACE · restringir a competicao

A loss do lote atual considera apenas classes presentes; o replay continua cobrindo todas as classes vistas. A meta e reduzir mudancas abruptas e interferencia assimetrica.

[Caccia et al. \(ICLR 2022\) · ER-ACE](#)

A-GEM · projetar o gradiente

Compara o gradiente atual ao gradiente de referencia da memoria. Se o produto interno for negativo, remove a componente conflitante antes do update.

[Chaudhry et al. \(ICLR 2019\) · A-GEM](#)

A mesma memoria pode servir para repetir exemplos, preservar logits ou construir uma restricao geometrica. O mecanismo - nao apenas o buffer - determina o comportamento.

EWC, SI e LwF preservam o passado sem guardar imagens

EWC

Fisher diagonal + penalidade quadrática em torno dos parâmetros consolidados.

[Abrir artigo](#)

SI

Importância sináptica acumulada pela contribuição $-\text{gradiente} \times \text{deslocamento}$.

[Abrir artigo](#)

LwF / distillation

O modelo anterior atua como professor sobre classes antigas; temperatura 2 no runner.

[Abrir artigo](#)

No Split-MNIST class-incremental, esses métodos mantiveram informação task-aware em graus distintos, mas não resolveram a calibração global sem replay.

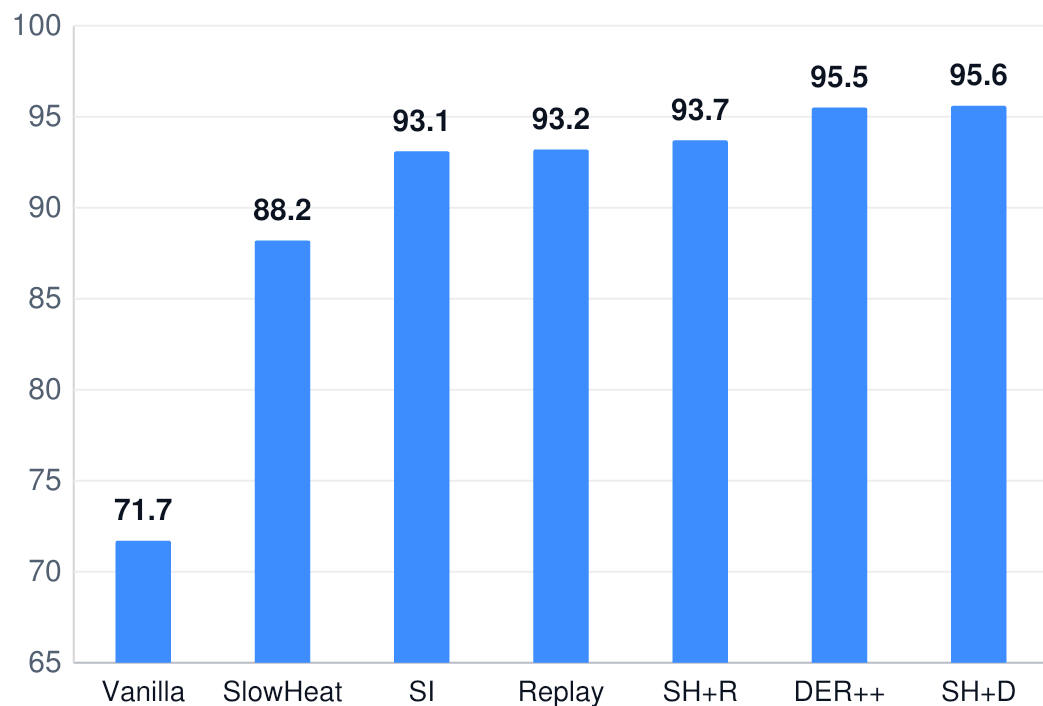
As ablacoes SlowHeat testam onde a protecao realmente ajuda

$\beta = 10 / 30 / 100$	forca crescente de protecao	adaptive	budget guiado por validacao
native_state	estado do AdamW segue o step nativo	unidirectional	protege apenas a linha de entrada
unbudgeted	remove a garantia de capacidade livre	none	wiring SlowHeat sem consolidacao
hard_freeze	mascara binaria exata	hidden	cabeca de saida fica plastica

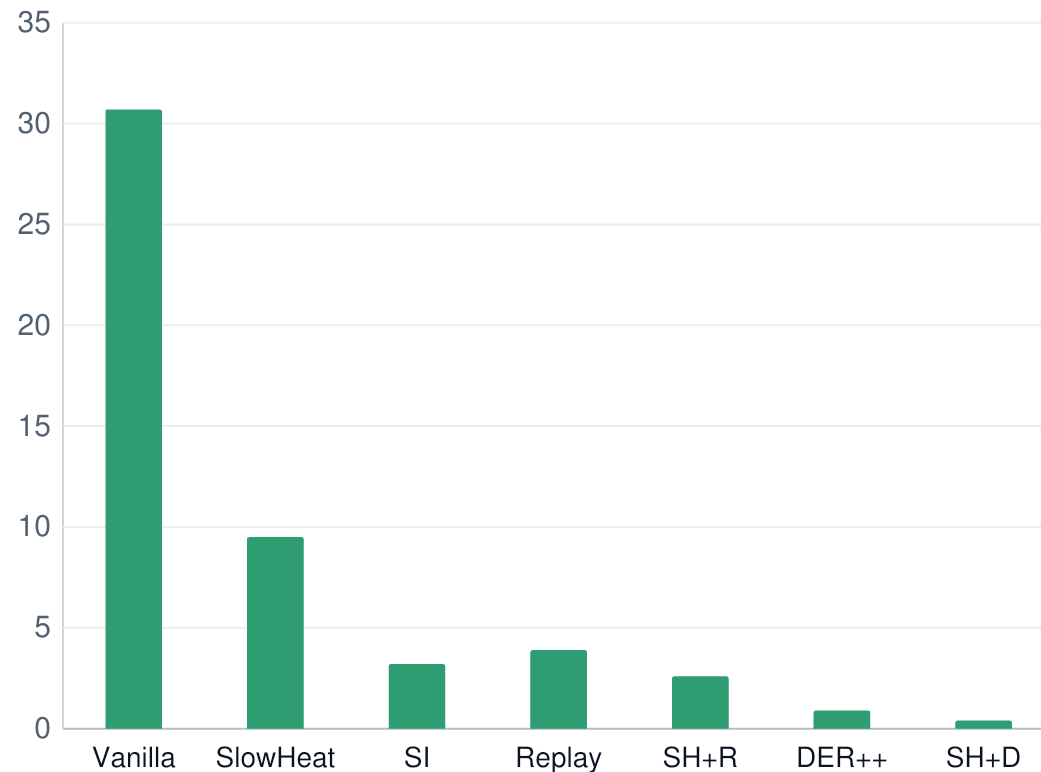
Ablacao nao e um novo artigo: cada nome estruturado combina escolhas locais sobre sinal, escopo, budget, estado do otimizador e metodo auxiliar.

Permuted-MNIST: DER++ domina a acuracia; SlowHeat reduz o forgetting

ACC final (%)



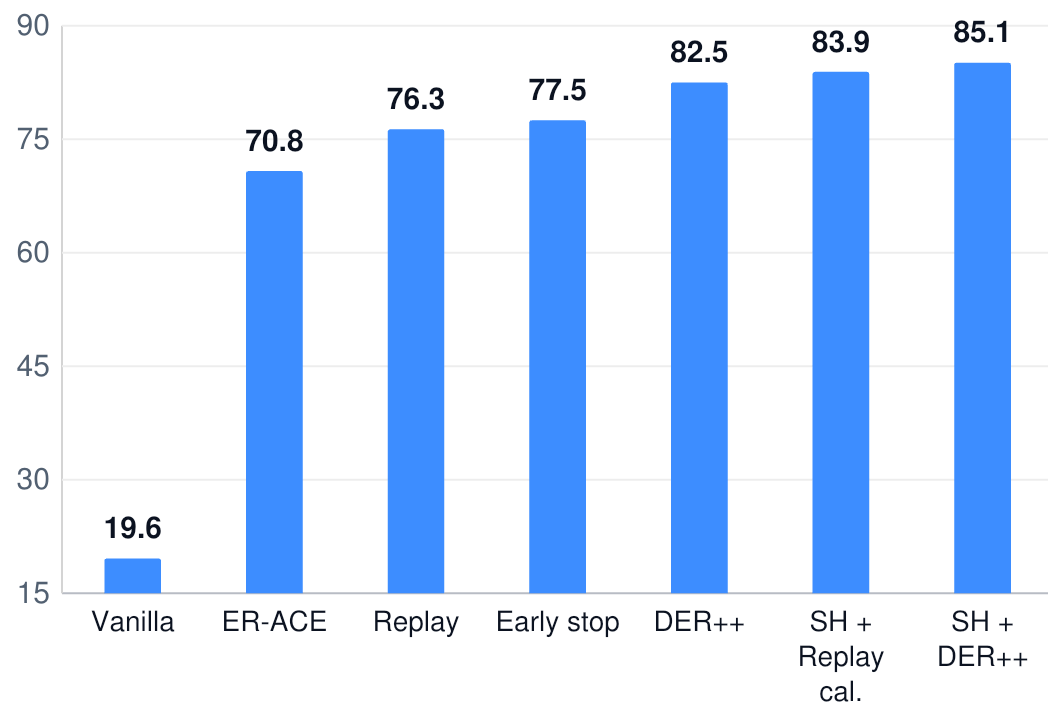
Forgetting (%)



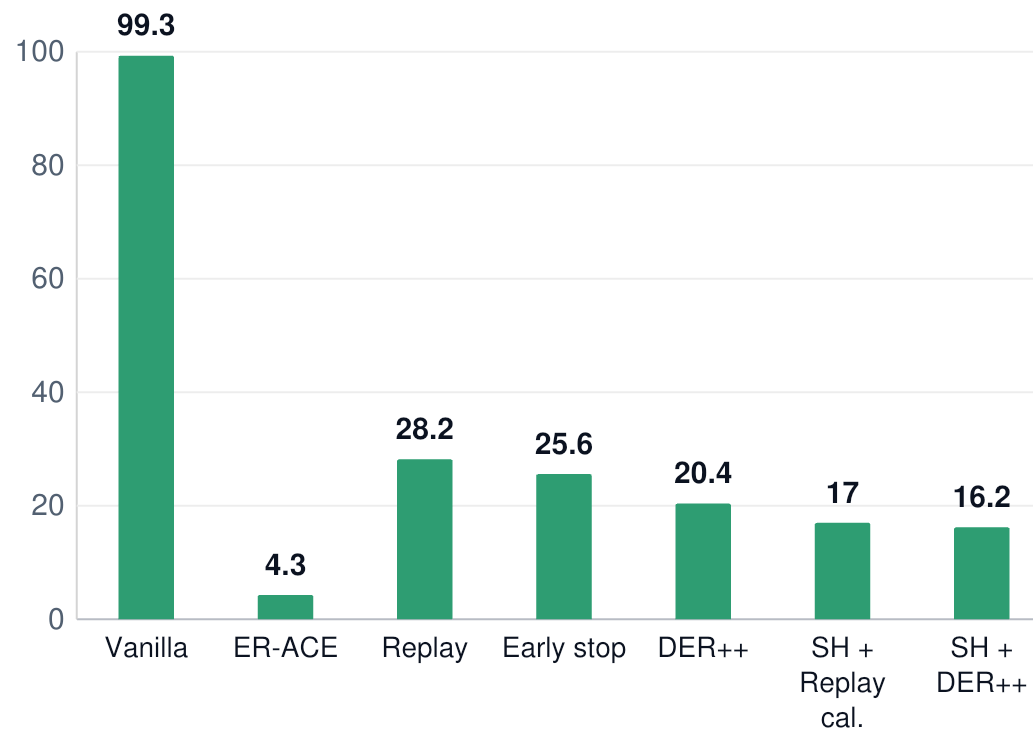
SH + DER++: 95,56% ACC e 0,38% forgetting · DER++: 95,52% e 0,94%.

Split-MNIST: SlowHeat + DER++ alcanza a maior acuracia media

ACC final (%)

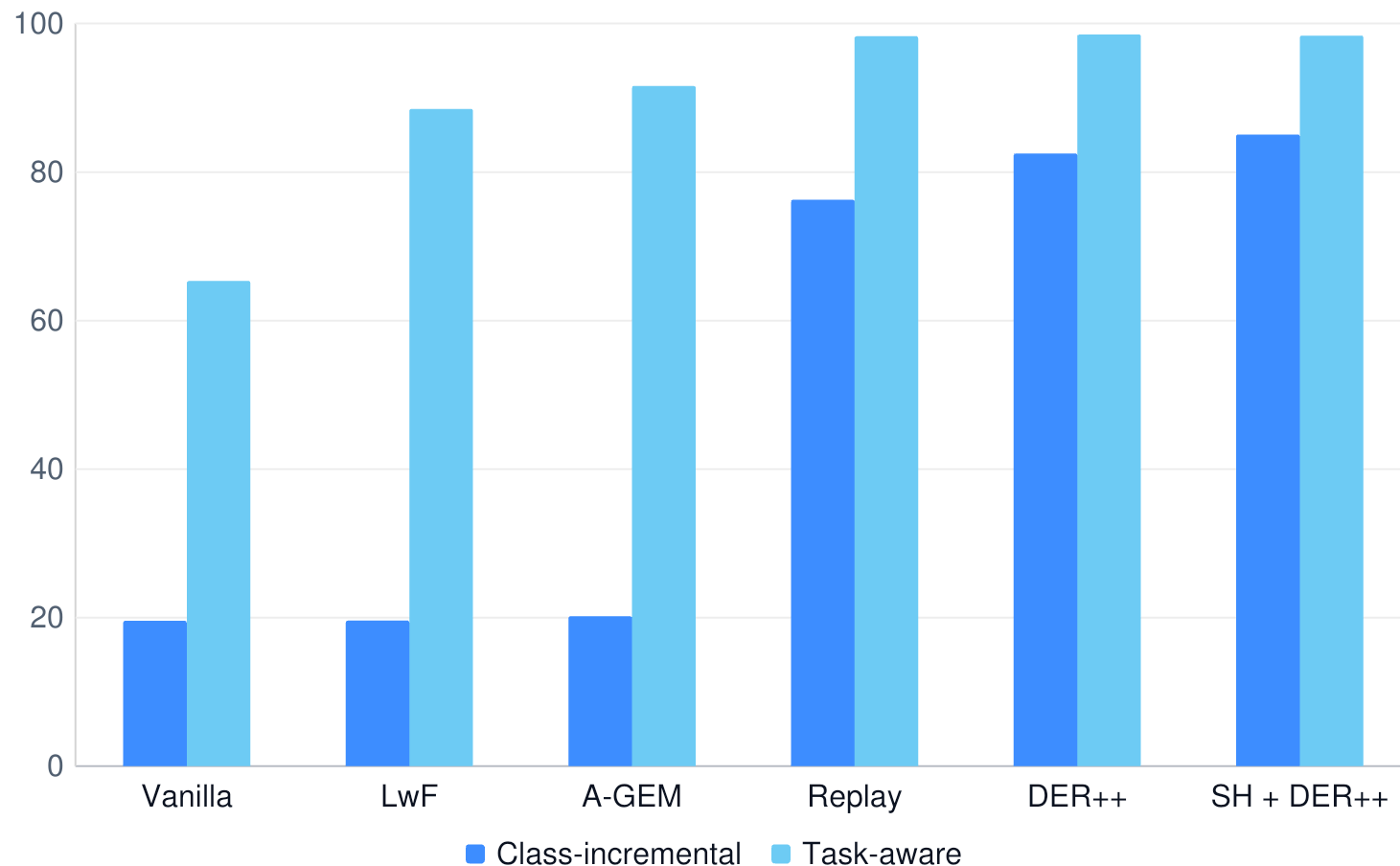


Forgetting (%)



SH + DER++: 85,06% ACC e 16,19% forgetting · DER++: 82,50% e 20,37%.

Classifier gap separa representacao de calibracao



A-GEM e LwF ainda distinguem classes dentro da tarefa quando recebem o task ID, mas falham na competicao global entre os dez logits.

71,41 p.p.

classifier gap do A-GEM

A cabeça global - nao apenas a representacao - e parte central do problema.

SlowHeat melhora acuracia, mas aumenta o tempo

Metodo	ACC	Tempo	FLOPs	Memoria
Replay + early stopping	77,46%	3,85 s	135,6 G	0,629 MB
Replay	76,27%	5,51 s	201,9 G	0,629 MB
DER++	82,50%	5,59 s	201,9 G	0,629 MB + logits
SlowHeat + DER++	85,06%	8,15 s	202,3 G	0,629 MB + logits

+45,8%

tempo: SlowHeat + DER++ vs.
DER++

+0,20%

FLOPs estimados no mesmo
contraste

A diferenca sugere overhead de snapshots,
estado e aplicacao de mascaras que a
contagem teorica de FLOPs nao captura.

Quatro conclusoes resistem aos dois benchmarks

01 Replay e essencial em Class-IL

SlowHeat isolado nao evita o colapso para ~20% no Split-MNIST.

02 DER++ e o baseline mais forte

Melhora claramente sobre Replay com custo e memoria adicionais pequenos.

03 SlowHeat funciona melhor como complemento

A combinacao com DER++ e o resultado mais promissor; com Replay simples, o ganho e pequeno.

04 A cabeca de saida importa

Hidden-only e calibracao de logits mudam fortemente o resultado class-incremental.

A proxima etapa e confirmacao pareada

10

seeds agregadas em cada
CSV

2

benchmarks MNIST
complementares

0

resultados Split-CIFAR versionados

- Reportar diferencas pareadas por seed, bootstrap/t de Student e tamanho de efeito.
- Pre-registrar uma confirmacao especifica de SlowHeat + DER++ contra DER++.
- Separar comparacoes por mesmas epocas, mesmos exemplos e mesmo custo observado.
- Validar em Split-CIFAR-10/100 e arquivar configuracao, matrizes, ambiente e hash do commit.

Controles e nucleo do SlowHeat

vanilla

Fine-tuning sequencial com AdamW; controle sem mecanismo de continual learning.

[AdamW](#)

slowheat

SlowHeat completo, $\beta=30$ e budget 0,25; protege tambem a saida por padrao.

[Contexto Taylor \(metodo local\)](#)

slowheat_beta_10

Mesma regra, protecao suave mais fraca: $m = 1/(1+10 \cdot \text{heat})$.

[Contexto Taylor \(metodo local\)](#)

slowheat_beta_30

Configuracao explicita equivalente ao SlowHeat padrao do runner.

[Contexto Taylor \(metodo local\)](#)

slowheat_beta_100

Protecao mais forte; privilegia estabilidade e reduz plasticidade efetiva.

[Contexto Taylor \(metodo local\)](#)

slowheat_adaptive

Atualiza o budget por sinal de aquisicao em validacao separada.

[Contexto Taylor \(metodo local\)](#)

slowheat_native_state

Mascara parametros, mas deixa momentos do AdamW seguirem o step nativo.

[AdamW](#)

Ablacoes estruturais do SlowHeat

slowheat_unidirectional

Remove a protecao fatorada downstream; protege apenas linhas de entrada.

[Contexto Taylor \(metodo local\)](#)

slowheat_unbudgeted

Remove a garantia de uma fracao minima de unidades totalmente plasticas.

[Contexto Taylor \(metodo local\)](#)

slowheat_none

Registra o mecanismo, mas nao consolida importancia; controle de wiring.

[AdamW](#)

hard_freeze

Converte a protecao em mascara binaria: unidades consolidadas ficam congeladas.

[Controle local · contexto EWC](#)

slowheat_hidden_beta_30_budget_0.25

SlowHeat apenas nas camadas ocultas, sem replay; cabeca livre.

[Contexto Taylor \(metodo local\)](#)

slowheat_replay_hidden_adaptive_beta_30_budget_0.25

Replay + hidden-only + budget adaptado por validacao.

[SlowHeat local + Replay](#)

Memoria, logits, restricoes e hibridos

replay

Mistura minibatches atuais e amostras de uma memoria episodica pequena.

[Tiny Episodic Memories](#)

distillation

Professor anterior preserva logits antigos, sem guardar imagens antigas.

[Knowledge Distillation](#)

slowheat_replay

Combina replay com SlowHeat completo, incluindo protecao da cabeca.

[SlowHeat local + Replay](#)

slowheat_distillation

Combina protecao funcional com professor anterior e distillation.

[SlowHeat local + Distillation](#)

derpp

CE atual + MSE de logits armazenados + CE sobre exemplos de replay.

[DER++](#)

slowheat_derpp_hidden_beta_30_budget_0.25

DER++ + SlowHeat nas camadas ocultas; cabeca permanece plastica.

[SlowHeat local + DER++](#)

Restricoes, regularizacao e Replay balanceado

er_ace

Restringe classes na loss atual e usa replay sobre todas as classes vistas.

[ER-ACE](#)

agem

Projeta o gradiente atual quando ele conflita com o gradiente da memória.

[A-GEM](#)

ewc

Fisher diagonal online + penalidade quadratica sobre parametros consolidados.

[EWC](#)

si

Importancia sinaptica acumulada ao longo da trajetoria de otimizacao.

[Synaptic Intelligence](#)

lwf_calibrated

LwF com pesos das losses definidos pela fracao de classes antigas e novas.

[Learning without Forgetting](#)

replay_balanced

Da peso 0,5 para loss atual e 0,5 para replay, independentemente do batch.

[Variacao local de Replay](#)

Políticas de treino e híbridos finais

replay_more_epochs

Executa 20 épocas por tarefa; controle de orçamento, não novo algoritmo.

[Replay](#)

replay_early_stopping

Até 30 épocas, paciência 3 e restauração do melhor estado de validação.

[Early stopping + Replay](#)

replay_global_lr_reduction

Reduz globalmente a LR por 1/31; controle para comparar seletividade.

[AdamW + Replay](#)

slowheat_replay_hidden_beta_30_budget_0.25

Replay + SlowHeat hidden-only, $\beta=30$ e pelo menos 25% de unidades livres.

[SlowHeat local + Replay](#)

slowheat_replay_partial_output_beta_30_budget_0.25

Replay com proteção parcial da camada de saída, em vez de totalmente livre.

[SlowHeat local + Replay](#)

slowheat_replay_hidden_beta_30_budget_0.25_calibrated

Adiciona offset local de logits para reduzir viés entre tarefas.

[SlowHeat local + Replay](#)

Referencias principais - links clicaveis

[Adam · Kingma & Ba \(2015\)](#)

[AdamW · Loshchilov & Hutter \(2019\)](#)

[Tiny Episodic Memories · Chaudhry et al. \(2019\)](#)

[DER++ · Buzzega et al. \(NeurIPS 2020\)](#)

[ER-ACE · Caccia et al. \(ICLR 2022\)](#)

[A-GEM · Chaudhry et al. \(ICLR 2019\)](#)

[EWC · Kirkpatrick et al. \(PNAS 2017\)](#)

[Synaptic Intelligence · Zenke et al. \(ICML 2017\)](#)

[Learning without Forgetting · Li & Hoiem \(ECCV 2016\)](#)

[Knowledge Distillation · Hinton et al. \(2015\)](#)

[Taylor saliency · Molchanov et al. \(ICLR 2017\)](#)

[Cenarios de continual learning · van de Ven & Tolias \(2019\)](#)

Functional SlowHeat, DualHeat e as combinacoes/ablacoes sao contribuicoes locais ainda sem artigo externo revisado por pares.

SlowHeat e promissor como complemento ao DER++ - ainda nao como melhoria geral confirmada.

Proxima decisao: confirmar o contraste pareado e validar fora do MNIST.

85,06% Split · 95,56% Permuted · n=10